HOWARD Tips

1 HOWARD Tips

How to hive partitioning into Parquet a VCF format file?

In order to create a database from a VCF file, process partitioning highly speed up futher annotation process. Simply use HOWARD Convert tool with `--parquet_partitions` option. Format of input and output files can be any avaialbe format (e.g. Parquet, VCF, TSV).

This process is not ressource intensive, but can take a while for huge databases. However, using `--explode_infos` option require much more memory for huge databases.

```
INPUT=~/howard/databases/dbsnp/current/hg19/b156/dbsnp.parquet
OUTPUT=~/howard/databases/dbsnp/current/hg19/b156/dbsnp.partition.parquet
PARTITIONS="#CHROM" # "#CHROM", "#CHROM,REF,ALT" (for SNV file only)
howard convert \
   --input=$INPUT \
   --output=$OUTPUT \
   --parquet_partitions="#CHROM" \
   --threads=8
```

How to process a huge file?

To process a huge file efficiently, you can partition it into multiple smaller Parquet files, process each partition individually, and then merge the results into a final output file. This approach helps manage memory usage and speeds up processing for large datasets. Below is a step-by-step explanation:

1. **Partition the Input File**:
   Use the `howard convert` command to split the input file into smaller Parquet files. This step ensures that the data is divided into manageable chunks for processing.
2. **Process Each Partition**:
   Loop through each Parquet file and perform the desired processing. For example, you can apply specific calculations or transformations to each partition. After processing, the original partition files need to be removed to perform merge.
3. **Merge Processed Files**:
   Combine all the processed Parquet files into a single output file using the `howard convert` command. This step consolidates the results into a unified file.

The following script demonstrates this workflow:

```
# Initialize variables
input_file=tests/data/example.vcf
output_file=/tmp/example.test.vcf
tmp_file=/tmp/example.test.partition.parquet
chunk_size=2 # default 1000000

# Step 1: Partition the input file into smaller Parquet files
howard convert --input=$input_file --output=$tmp_file --parquet_partitions='None' --chunk_size=$chunk_size

# Step 2: Process each Parquet file
for f in $tmp_file/*.parquet; do
    # Copy header for the split Parquet file
```

```
    cp $tmp_file.hdr $f.hdr
    # Perform processing (e.g., identifying variant type)
    howard process --input=$f --output=$f.processed.parquet --calculations="VARTYPE"
    # Remove the original split Parquet file
    rm -f $f $f.hdr
done
# Copy new header from last processed split Parquet file
cp $f.processed.parquet.hdr $tmp_file.hdr

# Step 3: Merge processed Parquet files into a final output file
howard convert --input=$tmp_file --output=$output_file

# Clean up temporary files
rm -rf $tmp_file
```

This method ensures efficient handling of large files while maintaining flexibility for various processing tasks.

How to hive partitioning into Parquet a huge VCF format file with all annotations exploded into columns?

Due to memory usage with duckDB, huge VCF file convertion can fail. This tip describe hive partitioning of huge VCF files into Parquet, to prevent memory resource crash.

The following bash script is spliting VCF by "#CHROM" and chunk VCF files with a defined size ("CHUNK_SIZE"). Depending on input VCF file, the number of chunk VCF files will be determined by the content (usually number of annotation within the VCF file). Moreover, Parquet files can also be chunked ("CHUNK_SIZE_PARQUET"). Default options This will ensure a memory usage around 4-5Gb.

Moreover, an additional partitioning can be applied, on one or more specific VCF column or INFO annotation: "None" for no partitioning; "REF,ALT" for VCF only with SNV (e.g. dbSNP); "CLINSIG" for ClinVar database (values such as "Pathogenic, Moslty-Pathogenic"...). Note that partitioning only works for small values (a value lenght will create a too long partition folder), and for not too many values for a column (partition fragments is maximum of 1024). This additional partition can take a while.

In order to create a high-performance database for HOWARD, INFO annotations can be exploded in columns. This option is slower, but generate a Parquet database that will be used by picking needed columns for annotation. Annotations to explode can be chosen with more options.

```
# Files
VCF=/tmp/my.vcf.gz                  # Input VCF file
PARQUET=/tmp/my.partition.parquet   # Output Partitioned Parquet folder

# Tools
BCFTOOLS=bcftools   # BCFTools
BGZIP=bgzip         # BGZip
HOWARD=howard       # HOWARD
TABIX=tabix         # Tabix

# Threads
THREADS=12          # Number of threads

# Param
CHUNK_SIZE=1000000000               # 1000000000 for VCF chunk size around 200Mb
CHUNK_SIZE_PARQUET=10000000         # 10000000 for parquet chunk size around 200Mb
PARQUET_PARTITIONS="None"           # "None" for no more partition, "REF,ALT" (SNV VCF) or "REF" or "CLNSIG".
CONVERT_OPTIONS=" --explode_infos " # Explode INFO annotations into columns

# Create output folder
rm -r $PARQUET
mkdir -p $PARQUET

# Extract header
$BCFTOOLS view -h $VCF --threads $THREADS > $PARQUET/header.vcf
```

```
# VCF indexing (if necessary)
if [ ! -e $VCF.tbi ]; then
    $TABIX $VCF
fi;

# For each chromosome
for chr in $($TABIX -l $VCF | cut -f1); do

    if [ "$chr" != "None" ]; then
        echo "# Chromosome '$chr'"

        # Create chromosome folder
        mkdir -p $PARQUET/#CHROM=$chr;

        echo "# Chromosome '$chr' - BCFTools filter and split file..."
        $BCFTOOLS filter $VCF -r $chr --threads $THREADS | $BCFTOOLS view -H --threads $THREADS | split -a 10
        nb_chunk_files=$(ls $PARQUET/#CHROM=$chr/*.gz | wc -l)

        # Convert chunk VCF to Parquet
        i_chunk_files=0
        for file in $PARQUET/#CHROM=$chr/*.gz; do

            # Chunk file to TSV and header
            ((i_chunk_files++))
            chunk=$(basename $file | sed s/.gz$//gi)
            echo "# Chromosome '$chr' - Convert VCF to Parquet '$chunk' ($PARQUET_PARTITIONS) [$i_chunk_files
            mv $file $file.tsv.gz;
            cp $PARQUET/header.vcf $file.tsv.gz.hdr;

            # Convert with partitioning or not
            $HOWARD convert --input=$file.tsv.gz --output=$file.parquet $CONVERT_OPTIONS --threads=$THREADS -

            # Redirect partitioninf folder if any
            if [ "$PARQUET_PARTITIONS" != "" ]; then
                rsync -a $file.parquet/ $(dirname $file)/
                rm -r $file.parquet*
            fi;

            # Clean
            rm -f $file.tsv*

        done;
    fi;
done;

# Create header
cp $PARQUET/header.vcf $PARQUET.hdr

# Show partitioned Parquet folder
tree -h $PARQUET
```

How to aggregate all INFO annotations from multiple Parquet databases into one INFO field?

In order to merge all annotations in INFO column of multiple databases, use a SQL query on the list of Parquet databases, and use `STRING_AGG` duckDB function to aggretate values. This will probably work only for small databases.

```
howard query \
   --explode_infos \
   --explode_infos_prefix='INFO/' \
```

```
   --query="SELECT \"#CHROM\", POS, REF, ALT, STRING_AGG(INFO, ';') AS INFO \
            FROM parquet_scan('tests/databases/annotations/current/hg19/*.parquet', union_by_name = true) \
            GROUP BY \"#CHROM\", POS, REF, ALT" \
   --output=/tmp/full_annotation.tsv

head -n2 /tmp/full_annotation.tsv
```

```
#CHROM  POS     REF     ALT     INFO
chr1    69093   G       T       MCAP13=0.001453;REVEL=0.117;SIFT_score=0.082;SIFT_converted_rankscore=0.333;S
```

How to explore genetics variations from VCF files?

CuteVariant: A standalone and free application to explore genetics variations from VCF files

Cutevariant is a cross-plateform application dedicated to maniupulate and filter variation from annotated VCF file. When you create a project, data are imported into an sqlite database that cutevariant queries according your needs. Presently, SnpEff and VEP annotations are supported. Once your project is created, you can query variant using different gui controller or directly using the VQL language. This Domain Specific Language is specially designed for cutevariant and try to keep the same syntax than SQL for an easy use.

Published in Bioinformatics Advanced - Cutevariant: a standalone GUI-based desktop application to explore genetic variations from an annotated VCF file

Documentation available on cutevariant.labsquare.org and GitHub

CuteVariant

How to generate dbNSFP databases?

dbNSFP is a database developed for functional prediction and annotation of all potential non-synonymous single-nucleotide variants (nsSNVs) in the human genome.

In order to use dbNSFP with HOWARD, databases need to be downloaded and formatted. The `databases` tool is able to download and generate VCF, Parquet and Partition Parquet format file.

```
# Download dbNSFP for ALL annotation, in VCF, Parquet and Partition Parquet, with INFO column
howard databases --assembly=hg19 --download-dbnsfp=~/howard/databases/dbnsfp/4.4a --download-dbnsfp-release='
```

To generate dbNSFP database in Annovar TXT format, because it can not be provided by Annovar tool, this following script is useful. Itcan be adapted dependgin on the dbNSFP release and needed assembly. For example, for release 4.4a of dbSNFP, and assembly 'hg19', the positional columns are '$8' for chromosome and '$9' for position (see dbNSFP doc).

```
# dbNSFP GZ to Annovar
# Script index annovar downloaded from https://github.com/WGLab/doc-ANNOVAR/issues/15. Official script does n
# Usage: perl annovar_idx.pl dbFile binSize
# Example: perl annovar_idx.pl hg19_clinvar_20160302.txt 1000 > hg19_clinvar_20160302.txt.idx

# Init
index_annovar=/path/to/annovar_idx.pl
table_annovar=/path/to/table_annovar.pl
dbSNFP_folder=/path/to/dbnsfp/4.4a/
vcf=/path/to/example.vcf.gz
annovar_file=hg19_dbNSFP44a.txt;
annovar_file_gz=$annovar_file.gz;

# Header
first_file=$(ls $dbSNFP_folder/dbNSFP4.4a_variant.chr*.gz | head -n 1)
nb_column=$(zcat $first_file | head -n1 | awk -F"\t" '{print NF}')
zcat $first_file | head -n1 | awk -v nb_column="$nb_column" -F"\t" '{gsub(/\r/,""); printf "#Chr\tStart\tEnd\
echo "Number of column: $nb_column"

# Format each variant file (by chromosome)
for file in $dbSNFP_folder/dbNSFP4.4a_variant.chr*.gz; do
    echo $file;
    zcat $file | grep "^#" -v | awk -v nb_column="$nb_column" -F"\t" '$8!="."{gsub(/\r/,""); printf $8"\t"$9"
```

```
done;

# Compress
gzip -9 -c $annovar_file > $annovar_file_gz &

# Index
perl $index_annovar $annovar_file 10000 > $annovar_file.idx

# Annovation test
$table_annovar $vcf . -protocol dbNSFP44a_annovar -buildver hg19 -out /tmp/myanno.vcf.gz -remove -operation f
```

Note: This script is not parallelized, not optimized